

14. SECURITY REPORT

The Security Report is one of the most important deliverables of this project. It provides a detailed analysis of the vulnerabilities identified during the security testing process. The report summarizes the findings obtained through manual security test execution and automated vulnerability scanning using OWASP ZAP.

The purpose of the Security Report is to identify security weaknesses, assess their potential impact, determine risk levels, and provide recommendations for improving application security.

During project execution, the Damn Vulnerable Web Application (DVWA) was tested for common web application vulnerabilities such as SQL Injection (SQLi) and Cross-Site Scripting (XSS). In addition, OWASP ZAP was used to perform automated vulnerability scans and generate security findings.

The results of the assessment revealed several security vulnerabilities intentionally present within DVWA, demonstrating the effectiveness of automated security testing techniques.

14.1 SQL Injection Findings

Overview

SQL Injection (SQLi) is one of the most critical web application vulnerabilities. It occurs when an application fails to properly validate user input before incorporating it into SQL queries. Attackers can manipulate SQL statements to gain unauthorized access to database information, bypass authentication mechanisms, or modify stored data.

During testing, the SQL Injection module of DVWA was evaluated using multiple malicious SQL payloads.

Test Payloads Used

The following payloads were executed during testing:

Payload 1

```
' OR '1'='1
```

Payload 2

```
1' OR '1'='1' --
```

Payload 3

```
admin' #
```

Payload 4

```
' UNION SELECT user,password FROM users --
```

Findings

The application failed to properly sanitize user input before executing database queries.

Observed behavior:

- Unauthorized records were retrieved.
- SQL queries were manipulated successfully.
- Sensitive database information was exposed.
- Input validation controls were absent or ineffective.

The application accepted malicious SQL payloads without performing adequate validation.

Impact

If present in a production environment, this vulnerability could allow attackers to:

- Access confidential data.
- Extract user credentials.
- Modify database records.
- Delete important information.
- Bypass authentication mechanisms.
- Gain administrative access.

SQL Injection is considered one of the most dangerous web application vulnerabilities because it directly affects the application's database.

Severity

Critical

Risk Level

High Risk

Evidence

During testing, SQL Injection payloads successfully returned unauthorized database records, confirming the presence of SQL Injection vulnerabilities.

14.2 XSS Findings

Overview

Cross-Site Scripting (XSS) occurs when an application accepts malicious scripts from users and executes them within a victim's browser. XSS vulnerabilities can be used to steal cookies, hijack sessions, redirect users, or execute malicious actions on behalf of legitimate users.

The XSS module of DVWA was tested using various JavaScript payloads.

Test Payloads Used

Payload 1

```
<script>alert('XSS')</script>
```

Payload 2

```
<img src=x onerror=alert('XSS')>
```

Payload 3

```
<svg onload=alert('XSS')>
```

Payload 4

```
<script>document.cookie</script>
```

Findings

The application failed to sanitize user-supplied input before rendering it within web pages.

Observed behavior:

- JavaScript code executed successfully.
- Browser alert messages appeared.
- User input was rendered without proper encoding.
- Client-side security controls were insufficient.

The successful execution of JavaScript payloads confirmed the existence of Cross-Site Scripting vulnerabilities.

Impact

If exploited in a real-world application, attackers could:

- Steal user session cookies.
- Hijack authenticated sessions.
- Redirect users to malicious websites.
- Modify page content.
- Capture sensitive information.
- Perform unauthorized actions.

XSS attacks can significantly compromise user security and application integrity.

Severity

Critical

Risk Level

High Risk

Evidence

Browser alert popups appeared after submitting malicious payloads, confirming successful script execution.

14.3 OWASP ZAP Findings

Overview

OWASP ZAP was integrated into the automation framework to perform automated vulnerability scanning.

The scanning process included:

- Spider Scan
- Active Scan
- Vulnerability Identification
- Risk Classification

The tool successfully discovered application pages and identified multiple security weaknesses.

Spider Scan Results

The Spider Scan successfully crawled the DVWA application and discovered:

- Login pages
- Vulnerability modules
- User-accessible URLs
- Form parameters
- Input fields

The application structure was mapped successfully.

Active Scan Results

The Active Scan identified several vulnerabilities within the application.

Finding 1: SQL Injection

Description:

The application was vulnerable to SQL Injection attacks.

Risk Level:

High

Status:

Confirmed

Finding 2: Cross-Site Scripting (XSS)

Description:

User-supplied input was executed as JavaScript within the browser.

Risk Level:

High

Status:

Confirmed

Finding 3: Missing Security Headers

Description:

Several HTTP security headers were not implemented.

Missing headers included:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options

Risk Level:

Medium

Status:

Confirmed

Finding 4: Cookie Without Secure Flag

Description:

Cookies were transmitted without the Secure attribute.

Risk Level:

Medium

Status:

Confirmed

Finding 5: Cookie Without HttpOnly Flag

Description:

Cookies could be accessed through client-side scripts.

Risk Level:

Medium

Status:

Confirmed

Finding 6: Information Disclosure

Description:

Application responses exposed unnecessary technical information.

Risk Level:

Low

Status:

Confirmed

OWASP ZAP Summary

OWASP ZAP successfully identified vulnerabilities across multiple categories and generated detailed security alerts with associated risk levels.

The tool proved effective in automating vulnerability discovery and security assessment activities.

14.4 Vulnerability Summary

The table below summarizes all vulnerabilities identified during testing.

Vulnerability	Severity	Risk Level	Status
SQL Injection	Critical	High	Confirmed
Cross-Site Scripting (XSS)	Critical	High	Confirmed
Missing Security Headers	Medium	Medium	Confirmed
Cookie Without Secure Flag	Medium	Medium	Confirmed
Cookie Without HttpOnly Flag	Medium	Medium	Confirmed
Information Disclosure	Low	Low	Confirmed

Vulnerability Distribution

Severity Level	Count
Critical	2
High	0
Medium	3
Low	1
Total	6

Key Observations

The most significant vulnerabilities identified during testing were:

- SQL Injection
- Cross-Site Scripting (XSS)

These vulnerabilities pose the greatest risk because they can directly impact application confidentiality, integrity, and availability.

The remaining vulnerabilities primarily relate to security configuration weaknesses and information disclosure issues.

14.5 Risk Assessment

Risk Assessment is the process of evaluating the likelihood and impact of identified vulnerabilities.

The purpose of risk assessment is to prioritize remediation efforts and determine which vulnerabilities require immediate attention.

Risk Rating Methodology

The following factors were considered:

Likelihood

The probability that a vulnerability can be exploited.

Impact

The potential damage resulting from successful exploitation.

Severity

The overall risk classification assigned to the vulnerability.

Risk Assessment Table

Vulnerability	Likelihood	Impact	Overall Risk
SQL Injection	Very High	Very High	Critical
Cross-Site Scripting (XSS)	High	High	Critical
Missing Security Headers	Medium	Medium	Medium
Cookie Without Secure Flag	Medium	Medium	Medium
Cookie Without HttpOnly Flag	Medium	Medium	Medium
Information Disclosure	Low	Low	Low

Risk Prioritization

Immediate Attention Required

The following vulnerabilities should be remediated immediately:

- SQL Injection
- Cross-Site Scripting (XSS)

These vulnerabilities directly expose sensitive information and provide opportunities for attackers to compromise the application.

Medium Priority Issues

The following vulnerabilities should be addressed in the next security update:

- Missing Security Headers
- Cookie Security Issues

These weaknesses increase exposure to other attack vectors.

Low Priority Issues

Information Disclosure findings should be reviewed and resolved to minimize information leakage.

Security Assessment Conclusion

The security assessment identified multiple vulnerabilities within the DVWA application. SQL Injection and Cross-Site Scripting vulnerabilities were successfully detected through automated test execution, while OWASP ZAP identified additional security weaknesses related to configuration and information disclosure.

The overall security posture of the application was determined to be **High Risk**, primarily due to the presence of Critical vulnerabilities such as SQL Injection and XSS. These findings demonstrate the importance of implementing secure coding practices, input validation mechanisms, output encoding techniques, and regular security assessments.

The results confirm that the automation framework successfully achieved its objective of identifying web application vulnerabilities and generating comprehensive security reports using Selenium WebDriver, TestNG, Maven, and OWASP ZAP.